

Schema-Aware Text-to-SPARQL

A Dual-Transformer Pipeline for Multi-Knowledge-Graph Question Answering

Datasets: LC-QuAD 1.0 (DBpedia) · LC-QuAD 2.0 (Wikidata)

How Previous Models Worked — Direct NL → SPARQL

Existing Text-to-SPARQL models learn to map a natural language (NL) question **directly to a SPARQL query** using a single model.

Example

1. Natural Language Question



When did Barack Obama become president of the USA?

2. Model (Trained End-to-End)

The model learns patterns from training data and generates the SPARQL query directly.

3. Generated SPARQL Query

```
SELECT ?startDate WHERE {  
  wd:Q76  wd:P39 wd:Q30 .  
  wd:Q76  p:P39 ?statement .  
  ?statement pq:P580 ?startDate .  
}
```

Problem with Existing Models

Single KG Only

Most models train on DBpedia or Wikidata only. Questions from outside their training KG fail completely.

Memorize URIs

Models memorize KG-specific identifiers (e.g. `wd:Q76` = Virat Kohli). Cannot **generalize** to new entities without retraining.

No Intermediate Reasoning

Direct NL→SPARQL generation is a black box. If the query is wrong, there is no way to understand why.

Our Approach — Schema-Guided Generation

here we trained model to Do not memorize KG identifiers. Instead, retrieve the correct schema elements at query time and inject them as input. The model learns to assemble SPARQL — not memorize URIs.

Example:

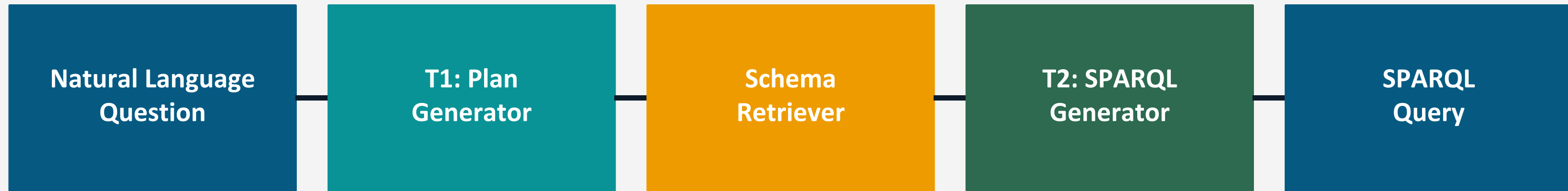
Question: When did Barack Obama become president of the USA?

Schema injected at query time:

Barack Obama → wd:Q76
president of USA → wd:Q30
position held → p:P39
start time → pq:P580

Model generates SPARQL using these – never memorized

Pipeline Architecture



Transformer 1 (flan-t5-xl)

Input: NL Question
Output: Structured plan marking entities, properties, classes
Trained on LC-QuAD 1 + 2 combined

Schema Retriever

DBpedia: Dense embedding + domain-range reranking
Wikidata: Dense embedding for props, API for classes
Entities: BM25 (DBpedia), Wikidata API (Wikidata)

Transformer 2 (flan-t5-xl)

Input: Question + Plan + Retrieved Schema
Output: Normalized SPARQL query
Trained on gold plan + gold schema

Transformer 1 — Plan Generator

What it does

- Converts NL question into structured plan
- Marks each step: entity / property / class
- Human-interpretable — no SPARQL knowledge needed
- KG-aware: [DBpedia] / [Wikidata] prefix tokens prevent forgetting old knowledge
- **base model** : flan-t5-xl (3b params)

Q: "How many films were directed by the director of Titanic?"

```
step1: find_entity
  surface_form: Titanic
  output_variable: ?titanic
```

```
step2: find_object
  property: director
  subject_variable: ?titanic
  output_variable: ?director
```

```
step3: find_subjects
  property: director
  object_variable: ?director
  output_variable: ?film
```

```
step4: aggregate
  function: COUNT
  target_variable: ?film
  output_variable: ?count
```

Schema Retriever — DBpedia

Step 1 — Dense Retrieval (Cosine Similarity)

Query label (e.g. "director") is encoded using *all-MiniLM-L6-v2* into a dense vector.

Cosine similarity computed against all DBpedia property/class embeddings.

Top-10 candidates returned.



Step 2 — Domain-Range Reranking

Each candidate property has a **domain** (subject type) and **range** (object type) from the DBpedia OWL ontology.

If the subject/object variables in the plan already resolve to a known type, candidates whose domain/range overlap get a **score boost**.

Domain-Range Reranking — Example

Query & Plan

Natural Language Question:

Which films have actors whose birthplace is *London*?

SPARQL Plan:

```
SELECT ?film WHERE {  
  ?film dbo:starring ?actor .  
  ?actor dbo:birthPlace dbr:London .  
}
```

?actor is the shared variable.

?actor connects both properties. So range of **starring** must match **domain** of **birthPlace**.

Gets Boosted

```
?film    starring    ?actor  
?actor   birthPlace ?city
```

Why boosted?

Range of `dbo:starring` = *Person*

Domain of `dbo:birthPlace` = *Person*

Match → Score Boost!

No Boost (Domain-Range Mismatch)

```
?film    starring    ?actor  
?film    foundingDate ?date
```

Why no boost?

Range of `dbo:starring` = *Person*

Domain of `dbo:foundingDate` = *Organisation*

No Match → No Boost

Instance Retriever — DBpedia

Step 1 — BM25 Candidate Retrieval

Surface form (e.g. "virat kohli") tokenised into unigrams.

BM25 scores the corpus of
1 M+ DBpedia labels for term overlap.

Why BM25?

Fast lexical matching handles exact and partial name matches efficiently across a large label corpus without needing dense embeddings at retrieval time.



Step 2 — Multi-Signal Re-ranking

Four signals combined per candidate:

BM25 base score	x1.0
String similarity	x0.2
Token coverage	x20.0
Length similarity	x5.0

Final step: Top-1 candidate URI returned and added to the schema for the 2nd transformer.

Schema Retriever — Wikidata

Properties — Dense Retrieval

Same approach as DBpedia — cosine similarity using all-MiniLM-L6-v2.

Key difference: Wikidata properties appear in 4 prefix forms:

wdt:P39 ← direct simple triple

p:P39 ← link to statement node

ps:P39 ← value inside statement

pq:P39 ← qualifier

Classes & Entities — Wikidata API

Wikidata Search API with fuzzy matching (~)
returns top-10 Q-ID candidates.

Classes — filtered by **P279 present** (subclass of):
"human" → P279 exists → wd:Q5

Entities — filtered by **P279 absent** (instance, not class):
"Virat Kohli" → P279 absent → wd:Q8192

No Domain-Range Reranking for Wikidata: Unlike DBpedia which has a well-defined OWL ontology with explicit domain/range declarations, Wikidata does not enforce strict domain-range constraints. Properties like P39 (position held) can apply to any entity type. Domain-range reranking is therefore only applied for DBpedia.

Transformer 2 — SPARQL Generator

Full Input to T2

`generate sparql [Wikidata]: Who is the mother of Albert Einstein?`

`plan: step1: action: find_entity | surface_form: Albert Einstein | output_variable: ?einstein | semantic_type: ENTITY ||`

`step2: action: find_statement | property: mother | subject_variable: ?einstein | output_variable: ?mother | semantic_type: PROPERTY`

`schema:`

`Albert Einstein → wd:Q937`

`mother → p:P25`

T2 Output (Generated SPARQL)

```
SELECT ?mother WHERE {  
  wd:Q937 p:P25 ?mother .  
}
```

Results — Transformer 1 (Plan Generator)

Training Config: flan-t5-xl | LoRA $r=16$ | $lr=5e-4$ | Effective batch=64 | 24k samples (DBpedia×2 + Wikidata)

91.95%

Token F1
(Overall)

60.89%

Step F1
(Overall)

61.69%

Step F1
DBpedia

60.73%

Step F1
Wikidata

30.33%

Exact
Match

Note: Token F1 measures token-level overlap — high score is expected as most tokens in a plan are correct even when one step is wrong. Step F1 is the stricter metric — the entire step must match exactly. 60.89% step F1 means ~61 out of 100 plan steps match gold exactly.

Results — Transformer 2 (SPARQL Generator)

1) Direct Results (Gold Schema — Exact URIs Provided)

92.11%	96.73%	91.20%	68.54%
Token F1 (Overall)	Token F1 DBpedia	Token F1 Wikidata	Exact Match

2) Results with Retriever (Schema Retrieval + Re-Ranking)

Metric	DBpedia (n=100)	Wikidata (n=100)	Overall (n=200)
Token F1	91.74%	87.25%	89.50%

Why Our Model Generalizes to New KGs

	Other Models	Our Model
Training KG	Single KG only	DBpedia + Wikidata combined
URI Handling	Memorized in weights	Retrieved at query time
New Entities	Fails — not in training	Entity linker handles at inference
New KGs	Must retrain	Plug in retriever — no retraining
Interpretability	Black box NL→SPARQL	Human-readable plan as intermediate step

Areas for Improvement & Future Work

Areas to Improve	Future Work
Schema Retriever Precision Retriever does not always return the exact correct property/class in top result. Retriever precision directly limits end-to-end pipeline accuracy. T1 Step F1 (60.89%) If planner generates wrong step type (e.g. find_object instead of find_statement), T2 receives incorrect structure. Improving step F1 cascades to better SPARQL. Train/Inference Mismatch T2 trained on gold schema but at inference gets retriever candidates. Fine-tuning T2 on retrieved schema will close this gap.	End-to-End Pipeline Evaluation Integrate retriever + T1 + T2 and measure true pipeline performance. Compare with gold schema numbers to quantify retrieval gap. Federated Queries Extend model to handle queries spanning multiple knowledge graphs simultaneously. More KGs Add Freebase/Yago support — plan vocabulary already supports CVT-style qualifier patterns.

Summary

Two-transformer pipeline: T1 generates interpretable plan, T2 generates SPARQL from plan + schema

Schema-guided approach: model never memorizes URIs — retrieves them at inference time

Multi-KG support: [DBpedia] / [Wikidata] prefix tokens prevent catastrophic forgetting

T1 achieves 91.95% Token F1 and 60.89% Step F1 on combined dataset

T2 achieves 92.11% Token F1 and 68.54% Exact Match (gold schema evaluation)